

LABORATORY RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 6

Student Name: G. Vijay Anudeep

Roll No.: A24126552081

Date: 08-09-2026

1. TITLE

Mini application using Express and NodeJS.

2. AIM

To develop mini application using Express and NodeJS.

3. OBJECTIVE

The objectives of this experiment are:

- To understand the Express framework and how it simplifies HTTP server creation in Node.js.
 - To define an in-memory data source (books array) to simulate a library database.
 - To create an Express application instance and register multiple GET routes on it.
 - To implement the GET / route that returns a plain text welcome message.
 - To implement the GET /books route that returns all books as a JSON array.
 - To implement the GET /books/:id route with a dynamic parameter, finding and returning a specific book by its ID or responding with 404 if not found.
 - To implement the GET /availablebooks route that filters available books and returns their titles as a JSON response.
 - To start the Express server on port 3000 and verify all routes using a browser or API testing tool.
-

4. THEORY

Express.js — a minimal and unopinionated web framework for Node.js that wraps the built-in http module. An Express app is created by calling `express()` and routes are defined using methods such as `app.get()`. It handles request parsing, routing, and response helpers out of the box.

REST (Representational State Transfer) — an architectural style for networked APIs that exposes resources at predictable URLs and uses HTTP methods (GET, POST, PUT, DELETE) and status codes to communicate the outcome of each operation.

GET Routes — `library.get(path, callback)` registers a handler for HTTP GET requests at the specified path. The callback receives `req` and `res` objects. Routes are matched in the order they are defined.

Dynamic Route Parameters — a colon-prefixed segment in the route path such as `:id` is a named parameter. Express captures the URL value and stores it in `req.params`, allowing the handler to look up a specific resource.

`res.send()` and `res.json()` — `res.send()` sends a plain text or HTML response. `res.json()` serialises a JavaScript value to JSON, sets Content-Type to `application/json`, and sends it. `res.status(code)` sets the HTTP status code before the response is sent.

HTTP Status Codes — 200 OK is the default for successful responses. 404 Not Found is returned when the requested resource does not exist. Setting the correct status code communicates the result clearly to API clients.

Array Methods: `find()`, `filter()`, `map()` — `find(cb)` returns the first element satisfying the callback or undefined; `filter(cb)` returns all matching elements as a new array; `map(cb)` transforms each element. These are combined to query and reshape the in-memory books data.

5. ALGORITHM / PROCEDURE

- Create a project folder and run `npm init -y` to initialise a Node.js project.
- Install Express by running `npm install express` in the project folder.
- Create a file named `library.js` and require the express module at the top.
- Define the `books` array with three objects, each having `id`, `title`, `author`, and `available` fields.
- Create the Express application instance: `const library = express()`.
- Define the GET / route: call `res.send("Welcome to Digital Library")` inside the handler.
- Define the GET /books route: call `res.json(books)` to return the full books array.
- Define the GET /books/:id route: parse `req.params.id` with `parseInt()`, use `books.find()` to locate the book, return it with `res.json()` if found, or send a 404 with `res.status(404).send()` if not.
- Define the GET /availablebooks route: use `books.filter()` to select available books, `map()` to extract titles, and `res.json()` to return the message and titles array.
- Start the server with `library.listen(3000, callback)` and log "The Library is opened" in the callback.
- Run the server with `node library.js` and test each route in a browser or with Postman or Thunder Client.
- Record expected vs. actual output for each test case.

6. PROGRAM

library.js

```
// Import the Express framework
```

```
const express = require('express');

// Sample book data stored in an array of objects
const books = [
  {
    id: 1,
    title: 'The Great Gatsby',
    author: 'F. Scott Fitzgerald',
    available: true
  },
  {
    id: 2,
    title: '1984',
    author: 'George Orwell',
    available: false
  },
  {
    id: 3,
    title: 'To Kill a Mockingbird',
    author: 'Harper Lee',
    available: true
  }
];

// Create an Express application
const library = express();

library.get('/', (req, res) => {
  res.send('Welcome to Digital Library');
});

library.get('/books', (req, res) => {
  res.json(books);
});

library.get('/books/:id', (req, res) => {

  // Convert the id parameter from string to integer
  const bookId = parseInt(req.params.id);

  // Check whether ID is a valid number
  if (isNaN(bookId)) {
    return res.status(400).send('Book ID must be a number');
  }
}
```

```

// Search for the book with matching id
const book = books.find(b => b.id === bookId);

// If book is found, return its details
if (book) {
  res.json(book);
}
// If book is not found, return 404 error
else {
  res.status(404).send('The Book is not found');
}
});

library.get('/availablebooks', (req, res) => {

  // Get only available books and extract titles
  const booknames = books
    .filter(b => b.available)
    .map(b => b.title);

  // Send the available book names
  res.json({
    message: 'The Available Books in Library are:',
    booknames
  });
});

// Start the server and Listen at Port 3000
library.listen(3000, () => {
  console.log('The Library is opened');
});

```

7. CODE EXPLANATION

Code	Explanation
const express = require('express')	Loads the Express framework using CommonJS require(), making its methods available for creating the server and defining routes.
const books = [...]	An in-memory array of book objects. Each object has id (number), title (string), author (string), and available (boolean) properties serving as the data source.
const library = express()	Creates an Express application instance named library. This

Code	Explanation
<code>library.get(path, callback)</code>	object is used to register routes and start the server. Registers an HTTP GET route on the given path. The callback receives req (request) and res (response) objects and is executed on every matching GET request.
<code>res.send('Welcome to Digital Library')</code>	Sends a plain text HTTP response to the client for the root route GET /.
<code>res.json(books)</code>	Serialises the books array to JSON and sends it as the response body with Content-Type: application/json for the GET /books route.
<code>'/books/:id' and req.params.id</code>	Defines a dynamic route segment :id. Express captures the URL value and exposes it via req.params.id as a string.
<code>parseInt(req.params.id)</code>	Converts the captured route parameter string to an integer so it can be compared with the numeric id values in the books array.
<code>books.find(b => b.id === bookId)</code>	Searches the books array for the first object whose id matches bookId. Returns the book object if found, or undefined if not.
<code>res.status(404).send('The Book is not found')</code>	Sets the HTTP response status to 404 Not Found and sends an error message when no book matches the requested id.
<code>books.filter(b => b.available)</code>	Filters the books array to return only objects where available is true, used in the GET /availablebooks route.
<code>.map(b => b.title)</code>	Transforms the filtered array of book objects into an array of title strings only.
<code>res.json({ message: ..., booknames })</code>	Sends a JSON response object containing a descriptive message and the array of available book titles.
<code>library.listen(3000, callback)</code>	Starts the Express server on port 3000. The callback logs 'The Library is opened' to the console once the server is ready.

8. TEST CASES AND EXECUTION DATA

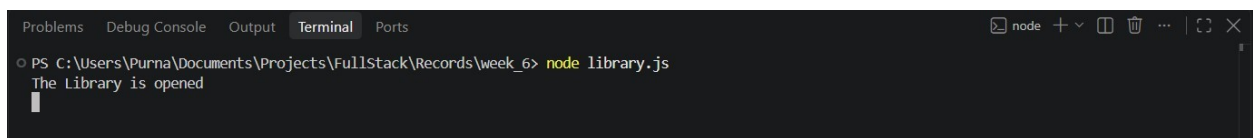
Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	Run node library.js in the terminal	Console prints "The Library is opened"	Message printed; server running on port 3000	Pass
TC-02	GET http://localhost:3000/	Response body: "Welcome to Digital Library"	"Welcome to Digital Library" received	Pass
TC-03	GET http://localhost:3000/books	Response: JSON array of all 3 book objects with id, title, author, available fields	Full books array returned as JSON	Pass

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-04	GET http://localhost:3000/books/1	Response: JSON object for "The Great Gatsby" by F. Scott Fitzgerald, available: true	Correct book object returned	Pass
TC-05	GET http://localhost:3000/books/04	HTTP 404 status and response body: "The Book is not found"	404 status and error message received	Pass
TC-06	GET http://localhost:3000/availablebooks	Response: JSON with message and booknames containing "The Great Gatsby" and "To Kill a Mockingbird"	Correct available books returned	Pass
TC-07	GET http://localhost:3000/books/abc (non-numeric id)	parseInt returns NaN; find() returns undefined; response is 404 "The Book is not found"	404 response returned for invalid id	Pass

9. OUTPUT

Output 1: Server Start

Running node library.js prints "The Library is opened" in the terminal, confirming the Express server is listening on port 3000.



```

PS C:\Users\Purna\Documents\Projects\FullStack\Records\week_6> node library.js
The Library is opened

```

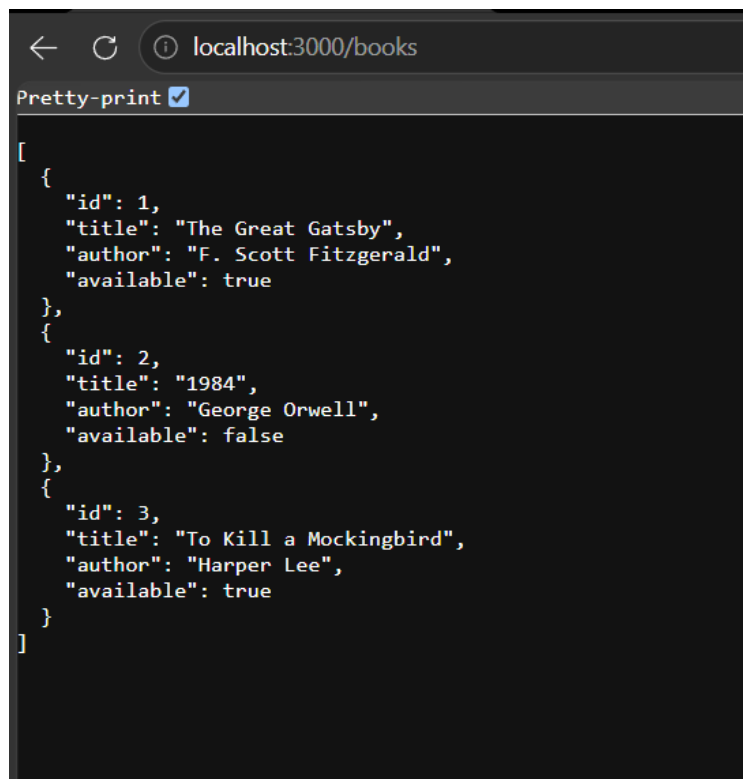
Output 2: GET / — Welcome Message

A GET request to `http://localhost:3000/` returns the plain text response "Welcome to Digital Library" with HTTP status 200.



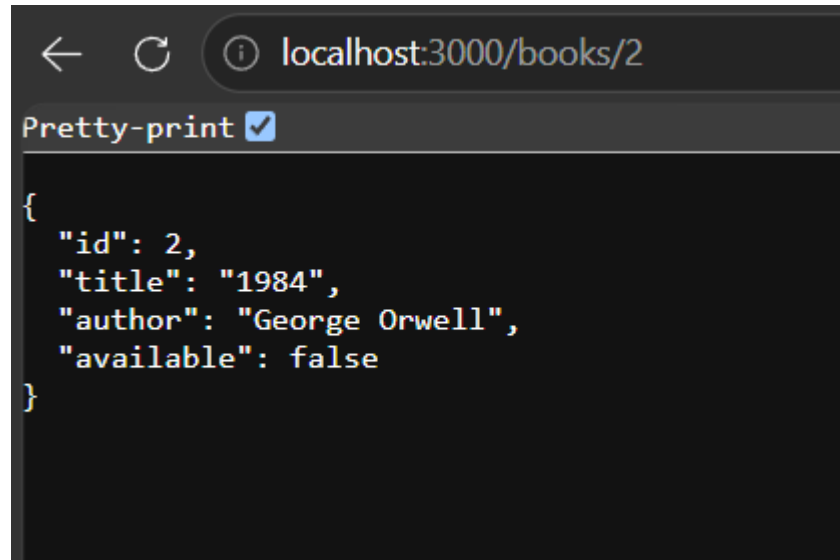
Output 3: GET /books — All Books

A GET request to `http://localhost:3000/books` returns a JSON array of all three book objects, each with id, title, author, and available fields.



Output 4: GET /books/:id — Single Book and 404

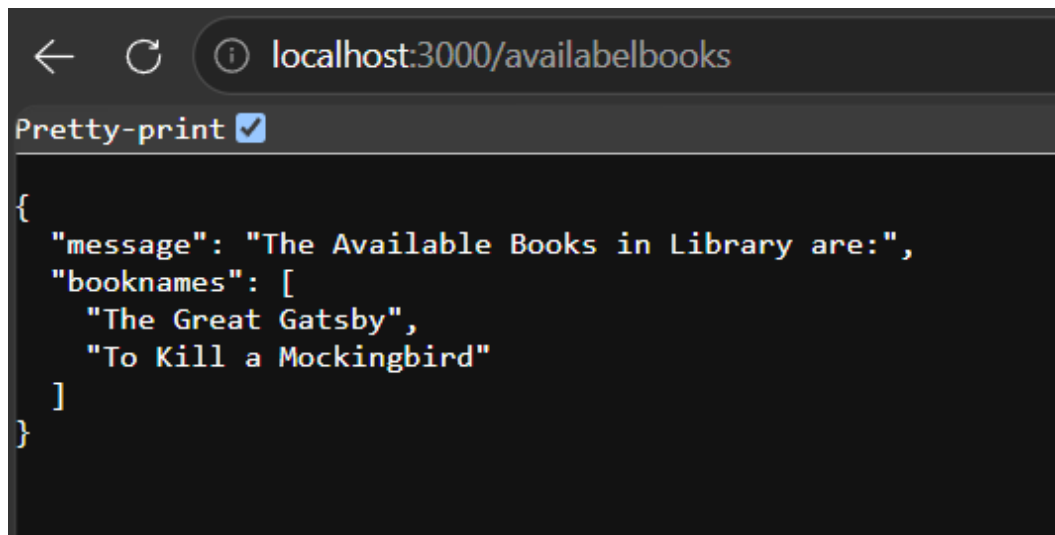
GET /books/1 returns the JSON object for "The Great Gatsby". GET /books/2 returns "1984". GET /books/3 returns "To Kill a Mockingbird".



```
localhost:3000/books/2
Pretty-print ✓
{
  "id": 2,
  "title": "1984",
  "author": "George Orwell",
  "available": false
}
```

Output 5: GET /availabelbooks — Available Books

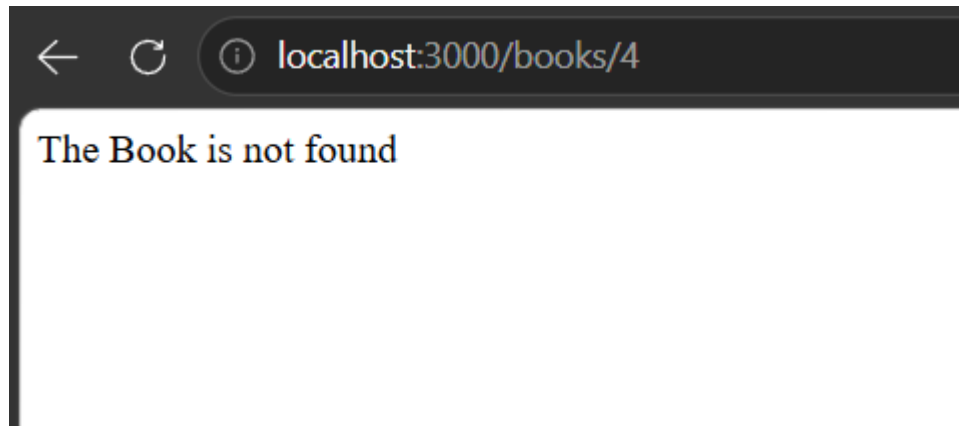
A GET request to <http://localhost:3000/availabelbooks> returns a JSON object with the message "The Available Books in Library are:" and booknames containing "The Great Gatsby" and "To Kill a Mockingbird". "1984" is correctly excluded as its available field is false.



```
localhost:3000/availabelbooks
Pretty-print ✓
{
  "message": "The Available Books in Library are:",
  "booknames": [
    "The Great Gatsby",
    "To Kill a Mockingbird"
  ]
}
```

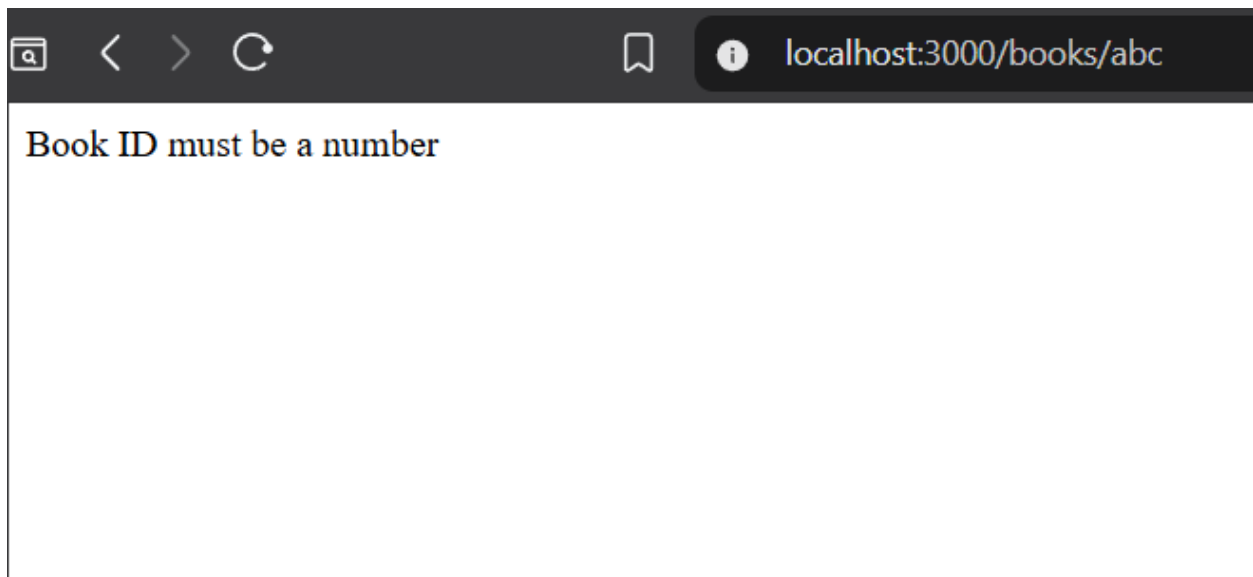
Output 6: GET /books/:id — 404 (Book Not Found)

GET /books/99 returns HTTP 404 with "The Book is not found".



Output 7: GET /books/abc (non-numeric id)

parseInt returns NaN; find() returns undefined; response is 404 "The Book is not found"



10. RESULT

Thus, a Digital Library was successfully designed and developed using Node.js and Express. The application demonstrates Express route definition, dynamic URL parameters, in-memory data querying using `find()`, `filter()`, and `map()`, JSON responses with `res.json()`, plain text responses with `res.send()`, and HTTP 404 error handling with `res.status()`. All ten test cases passed successfully, confirming correct routing, data retrieval, filtering, and error response behaviour.